

Зачем роботу отдельный контроллер

М6 • 25.02.2027 - 22.03.2027 • 75 часов

Результат месяца

Ты создашь контроллер ESP32-S3, который опрашивает IMU и энкодер, передаёт согласованные наборы измерений, ограничивает управляющий сигнал и при отказе переходит в заранее заданное, проверенное состояние. Это основа электроники ноги и пальца: компьютер может задержать сообщение, а локальный контроллер должен понимать, свежи ли измерения и разрешено ли продолжать работу. Итог включает прошивку, схему подключения, трассы шин, измерения времени цикла и протокол внесённых неисправностей. Для завершения модуля нужно подтвердить работу контроллера на стенде.

Что взять из предыдущих месяцев

Используй формат телеметрии и обработку ошибок M1, основы схем и измерений из электротехнических модулей, CAD-модель ноги M5 для размещения датчиков и кабелей. Входная проверка: объясни напряжение относительно общей земли, отличие входа от силового выхода, назначение подтяжки и единицы угла энкодера. Найди в документации именно своей платы схему питания и выводы, занятые USB, памятью и загрузочной конфигурацией. Если это пока не получается, включи разбор в первые 2 часа, до подключения внешних цепей.

Минимальная физическая часть определяется уже доступными компонентами: ESP32-S3, один совместимый IMU, источник угла либо энкодер, кнопка, защищённый маломощный индикатор выхода и средства измерений. Сначала проверь конкретные модели; компоненты нужно подбирать по их характеристикам. Недостающий датчик временно замени генератором пакетов, сохраняя тот же интерфейс. Такая замена доказывает логику программы, но в итоговом отчёте аппаратный пункт остаётся открытым до измерения на реальном устройстве.

Границы и связь с роботами

В М6 не требуется вращать нагруженную ногу или собирать весь робот. Выход разрешения проверяется на маломощной нагрузке; моторный стенд и его пределы относятся к М7. Для руки те же механизмы обнаружат пропавшую обратную связь пальца, для шагающей платформы — прекращение обновления угла и IMU. Программный запрет команды и независимое снятие энергии являются разными проверками. Схема остановки должна учитывать, что снятие питания с реальной вертикальной оси может вызвать падение; здесь проверяется её электрическая логика на стенде.

Подготовка результатов проверок для G1 входит в текущую работу. Сам интеграционный обзор G1 имеет отдельные 35 часов в общем плане и не включается в эти 75 часов. Все аппаратные лимиты берутся из документации выбранных компонентов и расчёта нагрузки; учебные частоты ниже служат измерительным сценарием, а не заявлением о безопасном режиме будущего робота.

Организация 75 часов и рабочего стенда

Четыре рабочих блока

Блок А, часы 1-20: 10 часов теории GPIO, PWM, ADC, таймеров и памяти; 10 часов практики подключения, измерения сигнала и прерываний. Блок Б, 21-40: 10 часов протоколов и 10 часов IMU, энкодера и устойчивого обмена. Блок В, 41-60: 10 часов планирования задач и 10 часов измерения колебаний времени цикла (джиттера), проверки очередей и сторожевого таймера (watchdog). Блок Г, 61-75: 5 часов анализа отказов и 10 часов интеграционных испытаний. Получается ровно 35 часов изучения и анализа плюс 40 часов практики.

Каждые 10 часов теории разбей на пять занятий по 2 часа: 35 минут чтения нужного раздела, 35 минут рисунка или расчёта, 40 минут малого проверочного примера, 10 минут заметок. Десятичасовые практикумы планируй на воскресенья; дели их на этапы с перерывами, а 10 часов учитывай как чистое рабочее время. Последние 5 часов анализа раздели на 2+2+1. Календарные границы модуля могут совпадать с соседними; отдых, отпуска и G1 сохраняются по общему плану. Выполняй пронумерованные шаги в свободные учебные часы без двойного учёта часов.

Структура результата

Заведи `firmware/` для ESP-IDF, `host/` для декодера Python, `config/` для распиновки и ограничений, `tests/` для кадров и переходов состояний, `evidence/` для трасс, `plots/` для графиков. В README укажи модель платы, ревизию, версию SDK, команды сборки и прошивки, схему внешнего питания и тип каждого датчика. Сохраняй исходные измерения отдельно от обработанных графиков: иначе нельзя понять, ошибка возникла в устройстве или в скрипте. Для каждого запуска запиши `run_id`, хеш прошивки, конфигурацию, время начала и вид загрузки.

Правило одного изменения

За один аппаратный эксперимент меняй один параметр: частоту шины, период опроса, глубину очереди или нагрузку логирования. До начала запиши ожидаемый эффект и условие завершения. После опыта запиши фактическое число кадров, ошибок и нарушений сроков выполнения (дедлайнов). Если трасса непонятна, вернись к последнему воспроизводимому варианту. Нельзя одновременно поменять провода, прошивку и скорость обмена, а затем объявить причину найденной. Используй отдельные коммиты для исходной рабочей версии и намеренного внесения отказа.

Первую фотографию подключения сделай с подписанными землёй, питанием, линиями данных и выходом разрешения. В журнале отдельно помечай «физически измерено», «проверено программным тестом» и «ожидается проверка на оборудовании». Так при задержке компонентов можно продолжить работу, а затем выполнить оставшиеся аппаратные проверки.

GPIO, PWM и ADC: сигналы настоящего робота

Что изучить

Разбери режимы GPIO: вход, выход, открытый сток, подтяжка, активный уровень. Для PWM свяжи частоту, период и коэффициент заполнения; $duty$ = время высокого уровня/период. Для ADC различай сырой код, напряжение на выводе и физическую величину после датчика. Разрядность сама по себе не гарантирует точность: нужно знать допустимый диапазон входа, откалибровать канал и оценить шум. Прочитай таблицу выводов платы и ограничения GPIO в справочнике ESP-IDF GPIO; параметры PWM уточняй по справочнику ESP-IDF LEDC. Мотор и катушку нельзя считать обычной GPIO-нагрузкой.

Задача 1. Разрешение движения по кнопке

Цель — проверить выдачу операторского разрешения для локального контроллера сустава. Начальные условия: маломощный индикатор вместо привода и явно известная схема активного уровня. 1. Составь таблицу входов, состояний и выходов. 2. Настрой безопасное электрическое состояние до запуска приложения. 3. Реализуй подавление дребезга по времени, сохранив исходные события для анализа. 4. Выполни 30 нажатий и отпусканий, включая короткие нажатия и удержание кнопки. Сохрани результат — схема, журнал и тест автомата. Проверка: одно подтверждённое действие на одно нажатие, отсутствие случайного разрешения при старте. Отказ: отключённый провод кнопки не должен восприниматься как новая команда запуска.

Задача 2. Проверка управляющего импульса

Цель — проверить, соответствует ли программная команда физическому сигналу. Используй только защищённый тестовый выход. 1. Выбери частоту и разрешение по ограничениям периферии. 2. Задай несколько значений коэффициента заполнения в пределах допустимого диапазона API. 3. Сними минимум 100 периодов каждого режима анализатором или осциллографом. 4. Посчитай средний период, минимум, максимум и отклонение $duty$. Сохрани трассу и таблицу команд и измеренных значений. Отдельно испытай нулевую команду и недопустимый параметр; драйвер должен сообщить отказ, а не молча оставить прежний активный выход.

Задача 3. Аналоговый канал диагностики

Задача готовит измерение напряжения питания или тока, но сначала использует известный допустимый тестовый сигнал. Запиши передаточную функцию канала и единицы. Собери серию при неизменном входном сигнале, рассчитай среднее и стандартное отклонение, затем измени сигнал минимум в трёх точках диапазона и сравни с независимым измерением. Сохрани коэффициенты преобразования, невязки калибровки и признак насыщения. При сигнале вне измеряемого диапазона программа помечает данные некорректными; она не подменяет их правдоподобным значением. Не подавай на ADC напряжение вне ограничений конкретной платы.

Справочник к опытам: [GPIO](#), [PWM](#). Выбери разделы настройки и ошибок API своей закреплённой версии ESP-IDF.

Таймеры, память и обработчики прерываний

Минимум теории для надёжной прошивки

Раздели аппаратный таймер, системный тик и измерение прошедшего времени. Периодический опрос должен планироваться от ожидаемого момента, иначе длительность работы добавляется к каждому периоду. Узнай, какие действия допустимы в ISR, как передать событие задаче и почему логирование или ожидание блокировки в обработчике увеличивают задержку. Для ESP-IDF сверяй актуальные варианты API в документации выбранной версии. Указатель на локальный массив нельзя использовать после завершения времени жизни этого массива; `volatile` не даёт атомарности и не заменяет синхронизацию между ядрами и задачами.

Задача 4. Метка времени события энкодера

Цель — связывать угол сустава с моментом измерения, а не со временем получения USB-пакета. Начни с генератора импульсов либо доступного энкодера. 1. Задай последовательность с известным числом фронтов. 2. В ISR сохрани минимальный набор: счётчик или отметку времени и уведомление задачи. 3. Обработай и передай данные вне ISR. 4. Сравни принятые события с исходной трассой. Результат — график интервалов, число потерянных событий и объяснение переполнения счётчика. Испытай увеличение частоты до обнаружения ошибки в пределах возможностей электрического стенда; полученное ограничение относится к этой конфигурации.

Задача 5. Кто владеет набором измерений

Цель — не допустить, чтобы из-за перезаписи памяти угол и показания IMU относились к разным моментам. Создай структуру `Snapshot` с `seq`, временем выборки, полями измерений и битами достоверности. 1. Сначала воспроизведи ошибку в тесте на компьютере: производитель данных (`producer`) меняет общий буфер, а потребитель (`consumer`) читает его позже. 2. Передавай снимок по значению через очередь либо используй буферы с явным владельцем. 3. Наполни поля связанными проверочными числами и проверь отсутствие смешанных наборов при замедлении потребителя. Сохрани тест целостности и описание времени жизни данных. Заранее определи поведение при переполнении очереди и добавь счётчик потерь.

Стек, куча и ограниченный ресурс

Зафиксируй объём очередей и оценку памяти на один снимок. Измерь минимальный запас стека задач доступными средствами SDK после испытания с наибольшей нагрузкой, а не только после старта. Проверь пути ошибки выделения памяти и повторной инициализации драйвера. Не добавляй динамическое выделение в частый цикл без причины; сначала используй ограниченные структуры. Для каждого таймаута укажи единицы и тип часов. Переполнение счётчика времени испытай искусственными значениями в тесте на компьютере; не жди его наступления на плате. Отчёт должен отличать наибольшую измеренную задержку от доказанной гарантии реального времени.

Справочник к опытам: [таймер](#), [энкодер](#). Выбери разделы настройки и ошибок API своей закреплённой версии ESP-IDF.

Шины и пакеты: IMU, энкодер, компьютер

Что пройти по интерфейсам

UART: скорость, формат символа, границы сообщений, полный и частичный кадр. I2C: адрес, операции с регистрами, ACK/NACK, подтяжки и таймаут. SPI: CS, CPOL/CPHA, порядок битов и чтение регистров. CAN и RS-485 изучи на уровне роли трансивера, топологии и согласования; арбитраж CAN и управление общим каналом RS-485 нельзя смешивать. Реализовать все пять физических шин за месяц не требуется: основной обмен с компьютером и один реальный сенсорный интерфейс обязательны, остальные закрепляются разбором трасс и небольшой схемой.

Задача 6. Узнать датчик по трассе

Цель — подтвердить, что данные поступают от нужного IMU и правильно расшифрованы. Исходные данные: техническое описание установленного датчика, регистр идентификатора и формат его измерений. 1. Проверь питание и логические уровни. 2. Считай идентификатор и настройки диапазона. 3. Сними транзакцию анализатором и вручную подпиши адрес, направление, данные и подтверждение. 4. Сопоставь байты с числом, учитывая знак и порядок байтов. Сохрани трассу с пояснениями и расчёт перевода исходного значения в физические единицы. Отказ: неверный адрес или разрыв линии приводит к завершению ожидания по таймауту и статус отсутствия данных, а не к бесконечному ожиданию.

Задача 7. Кадр телеметрии с восстановлением

Спроектируй кадр с ограниченной длиной: признак начала или кодирование границ, версия, длина, seq, timestamp, payload, контрольная сумма. Выбери алгоритм CRC из документированной реализации и укажи его параметры; простая сумма имеет другие свойства обнаружения ошибок. 1. Напиши кодирование и декодирование на компьютере. 2. Подавай поток по одному байту и случайными блоками. 3. Добавь пропущенный байт, мусор перед кадром, неверную длину и изменённую контрольную сумму. 4. Подключи тот же декодер к плате. Проверка: ни один повреждённый кадр из фиксированного набора не принимается, следующий целый кадр восстанавливается без перезапуска.

Пропускная способность и единицы

Оцени трафик до загрузки прошивки: байты кадра × кадров в секунду плюс накладные расходы выбранного транспорта. Сопоставь расчёт с фактической трассой и оставь запас под диагностику. Для каждого поля укажи SI-единицу, знак, диапазон и признак недостоверности invalid; не смешивай рад/с с град/с. У энкодера отдельно запиши число отсчётов на оборот (counts per revolution) с учётом режима декодирования и редуктора. Запрещено считать последнее число свежим только потому, что связь с компьютером всё ещё открыта: свежесть измерения проверяется по времени выборки и номеру в последовательности.

Справочник к опытам: [UART](#), [I2C](#), [SPI](#). Выбери разделы настройки и ошибок API своей закреплённой версии ESP-IDF.

FreeRTOS: измерение временных характеристик

Разделение обязанностей

Раздели работу на четыре функции: чтение датчиков, проверка состояния и разрешений, передача телеметрии, сервисная диагностика. Назначай приоритеты по последствиям задержки, а не по объёму кода. Сначала добейся предсказуемого выполнения с небольшим числом задач; закрепление по ядрам добавляй только после измерения. Прочитай отличия FreeRTOS в ESP-IDF и правила критических секций в ESP-IDF FreeRTOS: Critical Sections. Очередь передаёт данные, мьютекс (mutex) защищает общий ресурс, уведомление сообщает событие; выбор механизма зависит от задачи обмена и синхронизации.

Задача 8. Измерить период, задержку и дедлайн

Выбери учебный период T , например 10 мс для первого маломощного стенда, затем обоснуй его частотой датчика и допустимыми затратами времени на обмен. Плановый старт k -го цикла равен $t_0 + kT$. Запиши фактические моменты начала s_k и завершения e_k : джиттер старта $= s_k - (t_0 + kT)$, время работы $= e_k - s_k$. Просрочка есть, если завершение позже выбранного дедлайна. 1. Сними базовый запуск. 2. Повтори с интенсивным выводом в журнал и замедленным приёмником. 3. Сравни $p50$, $p95$, $p99$, максимум и число просрочек на одинаковом числе циклов. Сохрани исходные временные метки, конфигурацию и графики. Среднее не может скрывать редкий большой выброс.

Задача 9. Переполнение очереди без потери контроля

Цель — сохранить обновление состояния сустава при медленном обмене по USB. Сделай очередь фиксированной длины и два режима: полная запись данных для стендового опыта либо сохранение только свежего состояния для мониторинга. У каждого режима своя явно описанная политика потерь. Замедли потребителя данных так, чтобы очередь гарантированно заполнилась. Проверь максимальную занятость, `dropped_count`, порядок `seq` и время обнаружения устаревших данных. Прошивка не должна бесконечно ждать освобождения очереди в задаче, от которой зависит запрет выхода. Полученный предел сохраняй вместе с длительностью теста.

Задача 10. Проверка сторожевого таймера

Изучи контроль прерываний (interrupt watchdog) и задач (task watchdog) по разделу ESP-IDF Watchdogs. На маломощном стенде намеренно заблокируй выбранную задачу отдельной тестовой сборкой и зарегистрируй причину сброса. Перед сбросом и после загрузки измерь выход разрешения независимым каналом. Условие приёма: перезапуск не восстанавливает активное разрешение из сохранённой команды; нужна новая явная процедура запуска после успешных проверок. Обслуживание watchdog должно отражать работоспособность критических функций, а не выполняться безусловно сторонним таймером. Отдельно проверь остановку процесса на компьютере: это отказ связи, не обязательно отказ самой MCU.

Интегрированный стенд и автомат отказов

Спецификация controller-io v1

Входы стенда: IMU, угол или счётчик энкодера, команда оператора, датчики электрических ограничений при наличии. Выходы: снимок телеметрии, диагностические события и разрешение тестового канала. Начальный режим BOOT проверяет конфигурацию и переходит в DISARMED. Только явная команда запуска при выполненных условиях переводит в ARMED. Потеря достоверных измерений, превышение проверяемого лимита или E-stop переводят в FAULT. Снятие причины само по себе не переводит систему в ARMED. Возврат идёт через подтверждённый сброс отказа и DISARMED.

Задача 11. Таблица переходов и её проверка в коде

Исходные данные — список событий, состояний и физических выходов. 1. Для каждой пары «состояние — событие» укажи следующее состояние, код причины и разрешение канала. 2. Реализуй чистую функцию перехода, которую можно проверить тестами на компьютере. 3. Выполни команды в неверном порядке: запуск во время отказа, повторный сброс, возврат связи без подтверждения. 4. Сопоставь реальную телеметрию с таблицей. Сохрани таблицу, тесты и журналы минимум пяти последовательностей событий. Проверка: нет пути из BOOT или FAULT в ARMED без нужных условий и нового операторского действия.

Задача 12. Независимая остановка

Нарисуй путь энергии от источника до тестовой нагрузки и отдельно путь программной команды. Определи компонент, который способен прекратить подачу энергии независимо от обычного цикла MCU; выбирай его по документации нагрузки и схемы. Сначала проверь функцию на маломощном эквиваленте. Затем удержи программу в тестовом зависании и задействуй физическую остановку. Результат — схема, измерение выхода и протокол восстановления. Измерь время от физического события до прекращения тестового сигнала и сравни с заранее заданным требованием стенда. Такое испытание не присваивает самодельной системе промышленный класс безопасности.

Конфигурация и сброс при снижении питания

Храни конфигурацию с версией схемы и проверкой диапазонов. Не сохраняй режим ARMED как автоматически восстанавливаемое состояние. Для NVS изучи начальную инициализацию, запись и ошибки по ESP-IDF NVS. Испытай повреждённый или неизвестный формат на копии данных: загрузка с некорректными лимитами должна блокировать разрешение. Сброс по питанию проверяй управляемым отключением допустимого источника с маломощной нагрузкой, а не коротким замыканием. После возвращения питания журнал показывает причину перезапуска, конфигурация проверяется заново, разрешение движения отсутствует. Реальная устойчивость силовой части к просадкам проверяется уже с конкретным приводом M7.

Таблица испытаний и критерии проверки

Сбои входных данных

Тест F1: отключение IMU. Зафиксируй время последнего корректного снимка, момент таймаута и снятие разрешения. Порог свежести выбирается из ожидаемого периода обновления и допустимой задержки сенсора. Тест F2: испорченный кадр и ложная длина. Декодер отвергает пакет, не читает за границей буфера, увеличивает счётчик и восстанавливает следующий корректный кадр. Тест F3: зависшее значение энкодера при продолжающейся последовательности пакетов. Покажи разницу между «данные свежие» и «физическое измерение правдоподобно»; без модели движения неподвижность сама по себе не доказывает поломку.

Сбои выполнения и восстановления

Тест F4: получатель телеметрии не читает данные. Логирование теряет данные по выбранному правилу, а проверка свежести и отключение канала продолжают; измеряется задержка реакции. Тест F5: заблокирована контролируемая задача. Watchdog срабатывает в тестовой сборке, причина сброса сохраняется в отчёте, после загрузки контроллер остаётся в DISARMED. Тест F6: E-stop во время зависания MCU. Независимая цепь отключает тестовый выход; отпускание кнопки не запускает его снова. Для F5/F6 снимай физический выход, потому что текст «FAULT» в консоли не доказывает прекращения сигнала.

Границы конфигурации и времени

Тест F7: файл конфигурации содержит отрицательный период, неизвестную версию или нечисловой коэффициент. Каждая причина имеет понятную диагностику, конфигурация отклоняется целиком либо применяется по заранее описанным правилам атомарного обновления. Тест F8: перезапуск питания при ARMED на маломощной нагрузке. После старта требуется повторное разрешение. Тест F9: искусственное переполнение счётчика и два кадра с одинаковым seq. Счётчики статистики не выдают отрицательный возраст, повторы не маскируют отсутствие новых измерений. Тест F10: измерение ADC достигает края диапазона; программа сигнализирует насыщение, а не делает вывод о точной величине тока.

Как оформлять каждый результат

Для каждого сценария запиши: версию схемы и прошивки, исходное состояние, способ внесения отказа, ожидаемый переход, измеренные времена, имена файлов и итог: pass, fail или «не проверено физически». Повторяй критические переходы несколько раз и сохраняй разброс; один удачный сброс не характеризует надёжность. Для каждой временной метрики укажи начало, конец и источник времени, которым она измерена. Не вычитай метки ПК и MCU без синхронизации или оценки смещения. Если прибор недоступен, тест на компьютере остаётся полезен, но строку о физической задержке оставь незакрытой.

Порядок работы: часы 1-40

Часы 1-10: прочитать и спроектировать

1-2: опиши состав и параметры стенда, проверь распиновку, уровни и незанятые выводы. Нарисуй путь питания и начальное состояние выхода. 3-4: разбери сигналы GPIO и PWM, рассчитай период и заполнение, выбери измеримые режимы. 5-6: пройди ADC и перевод единиц, составь таблицу исходных и пересчитанных значений. 7-8: изучи таймеры и ISR, нарисуй временную диаграмму «событие — отметка времени — задача — пакет». 9-10: разберись с памятью снимков и очередями, напиши сценарий перезаписи общего буфера. К концу блока подготовь проект интерфейсов и список ожидаемых измерений.

Часы 11-20: первый практический блок

11-12: собери и проверь кнопку с защищённым индикатором, сохрани фото и таблицу состояний задачи 1. 13-14: выполни PWM-измерения задачи 2, включая некорректную конфигурацию. 15-16: собери аналоговые серии задачи 3 и построй график калибровки. 17-18: измерь фронты задачи 4 и вынеси обработку из ISR. 19-20: исправь владение снимком в задаче 5, добавь ограниченную очередь, сохрани исходную рабочую версию. Последние 15 минут каждого двухчасового шага отведи на оформление результатов; эти минуты входят в указанное время.

Часы 21-30: данные и протоколы

21-22: изучи UART и определение границ кадра, оцени требуемую пропускную способность. 23-24: разберись с I2C и регистрами конкретного IMU, вручную расшифруй пример транзакции. 25-26: изучи SPI и сравни режимы тактирования с техническим описанием датчика; не подключай новый датчик без проверки уровней. 27-28: составь сравнительную схему UART, I2C, SPI, CAN и RS-485: линии, адресация, разделение среды, трансивер и согласование. 29-30: разработай правила кадра, CRC и ресинхронизации; подготовь фиксированный набор повреждённых сообщений и ожидаемые ответы.

Часы 31-40: сенсорный тракт целиком

31-32: выполни чтение идентификатора и измерений IMU, проверь оси и знак. 33-34: сними и подпиши трассы задачи 6; сравни сырые значения и перевод в SI. 35-36: реализуй приём кадров задачи 7 с частичными чтениями. 37-38: внеси повреждения, отключение и возвращение датчика; убери бесконечные ожидания. 39-40: проведи длительный запуск в выбранном режиме, сверь число кадров и объём трафика с расчётом. Сохрани вторую исходную версию под именем baseline-2 и опиши её ограничения. Если нового оборудования нет, эти шаги выполняются с генератором тестовых данных и помечаются как программная часть, а не завершённый физический тракт.

Порядок работы: часы 41-75

Часы 41-50: время, ресурсы, отказ

41-42: прочитай модель задач FreeRTOS и различия ESP-IDF, нарисуй зависимости и выбранные приоритеты. 43-44: распредели доступное время одного цикла и определи метрики задачи 8; выбери частоту по датчику и доступному обмену. 45-46: сравни очереди, уведомления и mutex на своих данных, разбери случай медленного логирования. 47-48: изучи watchdog и причины сброса, сформулируй проверяемое поведение загрузки. 49-50: изучи сохранение конфигурации, раздели восстановление после сбоя питания на отдельные состояния. Не трать эти часы на оптимизацию кода до получения измерений.

Часы 51-60: измерительный практикум

51-52: реализуй запуск по периодическому расписанию и сохранение временных меток. 53-54: выполни исходный запуск задачи 8 и повтори его с нагрузкой, сформируй графики распределений. 55-56: выполни переполнение очереди задачи 9 и сравни выбранные политики. 57-58: внеси зависание задачи 10 в отдельной сборке, наблюдай физический выход при сбросе. 59-60: сверь повторный запуск, память и конфигурацию; устрани одну обнаруженную причину просрочек и повтори только затронутый опыт. К концу блока сформулируй измеренные пределы, длительность испытаний и то, что остаётся недоказанным.

Часы 61-65: пять часов анализа

61-62: проведи анализ опасностей стенда. Для каждой укажи источник энергии, событие отказа, возможное последствие, способ предотвращения и проверяемый признак его работы. 63-64: заверши таблицу автомата задачи 11 и схему независимого пути задачи 12; проверь, что аппаратная и программная остановки не зависят от одной и той же зависшей задачи. 65: подготовь последовательность приёмки, исходные состояния и формы протоколов. Это разделение 2+2+1 сохраняет общий баланс 35/40; дополнительных часов поверх программы не возникает.

Часы 66-75: заключительный стендовый блок

66-67: интегрируй автомат, проверку свежести измерений и проверку конфигурации. 68-69: выполни F1-F3 и F7, сравни сообщения с таблицей переходов. 70-71: выполни F4-F5 и проверь задержки под нагрузкой. 72-73: независимо проверь F6/F8 на маломощной нагрузке и восстановление; проверь полноту временных меток и исходных трасс. 74-75: воспроизведи проект из чистой сборки, сделай краткое демонстрационное видео и собери пакет G1. Если аппаратная проверка не выполнена, явно отметить это. Сокращать можно второй прототип транспорта, расширенный интерфейс ПК и дополнительные графики; схема отказов, основной сенсорный тракт и запрет самозапуска обязательны.

Приёмка, самопроверка и передача в М7

Условия завершения

Первое: прошивка собирается по инструкции и стартует без активного разрешения выхода. Второе: один реальный IMU и источник измерения угла имеют документированный формат, единицы и отметки времени; временные заменители явно обозначены. Третье: есть физические трассы основного обмена и проверка повреждённых кадров. Четвёртое: для рабочего цикла измерены период, джиттер, время выполнения и просрочки при описанной нагрузке. Пятое: watchdog, прекращение связи и E-stop не приводят к самозапуску после восстановления. Шестое: независимый путь остановки проверен по физическому выходу, а конфигурация ограничений содержит источник каждого значения.

Численные требования к частоте, таймаутам и времени снятия тестового сигнала нужно утвердить до эксперимента исходя из назначения стенда. В итоговой таблице расположи рядом требование, измерение, прибор, условия и заключение. Если результата не хватает, статус «не выполнено» полезнее среднего значения без описания его источника. Перед G1 подготовь схему с ревизией, фото, репозиторий, тесты, исходные измерения, графики, журнал неуспешных случаев и список незакрытых аппаратных предположений.

Вопросы для устного разбора

1. Почему volatile не решает гонку между ISR и задачей? 2. Чем возраст измерения отличается от возраста пакета? 3. Где должен возникнуть таймаут, если IMU не отвечает? 4. Почему CRC не доказывает физическую правдоподобность угла? 5. Что произойдёт с выходом во время работы загрузчика (bootloader)? 6. Почему watchdog нельзя обслуживать независимо от критических задач? 7. Как p99 может выглядеть хорошо при опасном одиночном выбросе? 8. Что именно отключает физическая остановка при зависании программы? 9. Какие данные допустимо восстановить из NVS? 10. Почему возвращение питания не равно разрешению продолжить движение?

На каждый вопрос отвечай через собственную схему, тест или трассу. Если ответ существует только в терминах библиотеки, придумай физический пример: пропало измерение угла пальца, нога получила старую команду, USB перегружен журналированием. Умение связать причину и наблюдаемый эффект является главным навыком этого месяца.

Что понадобится в следующем модуле

В М7 передаются работоспособный контроллер, интерфейс измерений и команды, перечень таймаутов, журнал сбросов, автомат отказов и схема независимой остановки. Добавление реального привода потребует новых токовых, тепловых и механических пределов; значения малоомощного стенда не переносятся автоматически. Готовый интерфейс поможет измерить задержку привода и воспроизвести идентификационный опыт. Сохрани выпуск controller-io-v1 и перечень необходимых аппаратных проверок отдельно от будущих изменений управления.

Литература: короткий маршрут чтения

Читай только указанные фрагменты перед соответствующим опытом. Литература преимущественно на английском; объяснения, словарь и отчёт веди по-русски. Чтение и подготовка входят в существующие 75 часов; целые книги проходить не требуется.

Основное чтение и границы переноса

1. Jonathan W. Valvano, Embedded Systems: Introduction to ARM Cortex-M Microcontrollers. Главы 4, 6, 9–11: ввод и вывод, указатели и структуры данных, прерывания и реальное время, аналоговые интерфейсы, связь. Для задач 1–7 читай принципы; описания регистров Cortex-M не применяй к ESP32-S3, а учебную плату TI не покупай ради книги.

2. Richard Barry / FreeRTOS, Mastering the FreeRTOS Real Time Kernel. Текущая онлайн-книга: главы 4, 5, 7: Task Management, Queue Management, Interrupt Management. Для задач 8–9 нужны состояния задач, очереди и отложенная обработка событий ISR. В старых PDF нумерация другая; ориентируйся на эти названия.

3. Edward A. Lee, Sanjit A. Seshia, Introduction to Embedded Systems, 2-е изд. (2017). Глава 3 Discrete Dynamics; глава 11 Multitasking. Выборочно: конечные автоматы и совместное выполнение задач. Для задачи 10 нарисуй переходы автомата отказов, а для очереди объясни гонку и владение данными. Формальную верификацию всего контроллера здесь не проходи.

Документация вместо догадок об API

ESP-IDF FreeRTOS. Tasks, SMP Scheduler, Critical Sections. После книги сравни именно реализацию ESP-IDF: единицы размера стека, ядра, блокировки. Версию справочника переключи на закреплённую версию проекта.

Отказ и восстановление. Watchdogs: IWDT, TWDT, JTAG и NVS: Read, Write and Erase; Error Handling. Для задачи 10 проверь зависание, перезапуск и повреждённую конфигурацию; ARMED не восстанавливается автоматически.

Шины и периферия. Ссылки на GPIO, LEDC, таймер, UART, I2C, SPI и PCNT приведены на соответствующих страницах практикума. Добавь техническое описание своего IMU: регистры, единицы, порядок байтов и условия доступа.

Итог чтения. Подготовь схему выводов, таблицу задач с периодами, схему владения буферами и автомат отказов. Каждый документ должен объяснять реальную трассу или воспроизводимый сбой.

Словарь: термины и определения

Микроконтроллер (MCU). Микросхема с процессором, памятью и периферией для непосредственного управления устройством; ESP32-S3 использует ядра Xtensa.

GPIO. Программируемый цифровой вывод общего назначения; режим, допустимое напряжение и функции задаются контроллером и схемой платы.

Прерывание (interrupt). Событие, вызывающее обработчик вне обычной последовательности задач; задержка обслуживания зависит от приоритетов и блокировок.

ISR. Обработчик прерывания с ограниченным временем работы и набором разрешённых операций; длительную обработку обычно передаёт задаче.

Таймер (timer). Аппаратный или программный механизм отсчёта времени; аппаратный счётчик и программный обработчик имеют разные источники задержки.

Джиттер (jitter). Колебания моментов запуска или длительности операции относительно выбранного эталона; измеряется во времени, например в микросекундах.

Дедлайн (deadline). Момент, к которому вычисление или действие должно завершиться; пропуск определяется требованиями системы, а не средней производительностью.

Задача RTOS (task). Независимый контекст исполнения со стеком и состоянием; планировщик выбирает готовую задачу с учётом приоритетов.

Очередь (queue). Ограниченный буфер передачи сообщений между контекстами; переполнение и ожидание должны иметь явно выбранную политику.

Гонка данных (data race). Несогласованный конкурентный доступ к общей памяти, включающий запись; результат может зависеть от порядка исполнения.

Критическая секция (critical section). Участок доступа к общему состоянию, защищённый согласованным механизмом синхронизации; его длительность влияет на задержки системы.

DMA. Передача данных между периферией и памятью без покомандного копирования процессором; требует согласования владения буфером и завершения передачи.

UART. Асинхронный последовательный интерфейс с согласованными скоростью и форматом символа; границы сообщений протокол задаёт отдельно.

I2C. Адресная синхронная шина данных SDA и такта SCL; электрическая реализация требует проверки подтяжек, ёмкости и уровней.

SPI. Синхронный последовательный интерфейс с тактом и выбором устройства; режим, порядок битов и границы транзакции согласуются с датчиком.

CRC. Контрольный код для обнаружения определённых ошибок передачи; не обеспечивает подлинность отправителя и защиту от намеренной подмены.

Возраст данных (data age). Время с момента получения физического измерения до его использования; включает передачу, очереди и обработку.

Сторожевой таймер (watchdog). Механизм, обнаруживающий отсутствие ожидаемого подтверждения работы; перезапуск контроллера не гарантирует безопасное состояние силовой части.

Конечный автомат (finite-state machine). Модель с конечным набором состояний и заданными переходами по событиям; удобна для разрешения работы и обработки отказов.

NVS. Энергонезависимое хранилище значений с API доступа; конфигурации нужны версия и проверка допустимости после загрузки.

Что купить, установить и проверить

Подготовку выполни внутри первых учебных блоков вместо повторного общего обзора. Общий объём остаётся 75 часов; ожидание доставки в него не входит. Статусы ниже обозначают твои действия, а не уже выполненные покупки или установки.

ИСПОЛЬЗОВАТЬ ИМЕЮЩЕЕСЯ И ДОКУПИТЬ ДО ПРАКТИКУМА

Повторно используй Mac M4, мультиметр, источник, макетную плату, проводку и маломощную нагрузку M04. **Купить: 2 платы [ESP32-S3-DevKitC-1](#)** одной проверенной ревизии: первая — контроллер, вторая — генератор импульсов и устройство для обмена пакетами. Сверх Flash, PSRAM, схему и выводы, занятые USB и загрузочной конфигурацией.

1 модуль IMU на плате: доступ к исходным данным трёхосевых акселерометра и гироскопа, документированные регистры, I2C и/или SPI, доступная линия прерывания, питание и логические уровни, совместимые с 3,3 В. Проверь поддерживаемый драйвер для ESP32-S3, выбранного интерфейса и версии ESP-IDF. Если такой датчик уже есть — не покупай второй. Сенсор с одними готовыми углами не заменяет доступ к сырым каналам.

1 логический анализатор: не менее 8 цифровых каналов, входы для выбранных уровней, декодеры UART, I2C и SPI и частота выборки с запасом для изучаемой шины. Пример — [Saleae Logic 8](#); до заказа проверь ПО для Apple Silicon. **1 комплект USB:** два кабеля передачи данных с разъёмами именно плат, переходник или хаб при необходимости, щупы с общей землёй и разъёмы.

УСТАНОВИТЬ СЕЙЧАС — macOS ARM

Начни с [PlatformIO IDE для VS Code](#): сборка, загрузка и монитор простого проекта. Затем [ESP-IDF, официальный Get Started](#) для целевой платформы esp32s3 с совместимыми CMake, Ninja и набором инструментов компиляции. Закрепи версии; версия платформы PlatformIO espressif32 и версия среды ESP-IDF — разные поля, их совместимость проверяется отдельно.

Добавь [KiCad для macOS](#) для схемы подключения и электрической проверки; ПО именно своего анализатора, например [Logic 2 для Saleae](#). Для совместимого с sigrok прибора может понадобиться [PulseView](#); поддержку ARM и USB проверь до покупки. Logic 2 не обслуживает произвольные анализаторы. Ubuntu 24.04 ARM64 в UTM используй лишь после отдельной проверки USB; основной путь сборки и измерений — нативный Mac.

ПРОВЕРКА УСТАНОВКИ

На обеих платах: build→flash→monitor, аппаратный сброс и чтение версии прошивки. Затем проверь петлевой обмен UART (loopback) или обмен между платами, измерь PWM, выполни одну транзакцию I2C или SPI с IMU и экспортируй трассу. Сверх единицы исходных отсчётов и монотонность времени. В KiCad открой схему и выполни ERC; сохрани версии среды и схему назначений выводов.

Необязательно, на будущее: STM32 — только для отдельного сравнения; мотор с энкодером приобретается в M07. В M06 источник угла может быть уже имеющимся энкодером, а вторая плата — генератором квадратурных импульсов. Генератор проверяет интерфейс; требуемые реальные измерения датчиков остаются отдельными пунктами приёмки.

Основы движения: угол, скорость и время

Обязательно: связь измерения с движением

Контроллер ноги получает отсчёты, но движение задаётся углом $q(t)$ и его изменением во времени. До опыта объясни: $90^\circ = \pi/2$ рад; положительное вращение определяется ориентированной осью сустава, а одинаковый угол в двух пакетах ещё не доказывает остановку. В М6 используй имеющийся датчик либо генератор второй ESP32-S3, без нового мотора.

Модель измерения и пример расчёта

Пусть c_k — развёрнутый счётчик без переполнения, C — число отсчётов на оборот именно измеряемого вала после выбранного декодирования, t_k — время выборки в секундах по одному источнику времени MCU. Относительный угол $q_k = 2\pi(c_k - c_0)/C$, в рад. Если энкодер на моторе, переход к углу звена требует отдельно известного передаточного отношения и знака. Здесь редукции нет.

Учебные данные: $C=4000$, $c=(100;104;110)$, $t=(0;0,010;0,025)$ с. Получаем $q=(0;0,006283;0,015708)$ рад. Интервальная скорость $v_k=(q_k - q_{k-1})/(t_k - t_{k-1})$ в обоих интервалах равна 0,628319 рад/с. Деление второй разности на номинальные 0,010 с даёт ложные 0,942478 рад/с: колебания интервала создают видимость ускорения.

Разностная скорость — средняя за интервал, не точная мгновенная производная. Один отсчёт соответствует 0,001571 рад; при коротком окне квантование заметно в скорости. Сглаживание уменьшает шум, но добавляет задержку. Время USB-приёма не заменяет время физической выборки: поздно пришедший неповреждённый пакет может содержать устаревший угол.

Один сквозной опыт задач 4, 5 и 8

Сначала воспроизведи таблицу на компьютере. Затем передай со второй платы известную последовательность A/B и сверь принятые фронты с логическим анализатором. Сохрани counts, sample_time, receive_time, seq, q и интервальную v. На графике сравни скорость по реальному и номинальному интервалам. Постоянная добавка ко всем c не должна менять относительные углы.

Повтори с задержкой чтения у потребителя и одинаковой меткой времени у двух кадров. Нулевой или отрицательный интервал отклоняется до деления; просроченный снимок не получает статус свежего из-за нового пакета. Например, при постоянной $v=0,628319$ рад/с возраст 0,05 с означает изменение угла на 0,031416 рад. Это масштаб ошибки, а не назначенный таймаут робота.

Встроено в 75 часов: 1 час из 7-8 — модель времени; 1 час из 17-18 — задача 4; 1 час из 53-54 — графики задачи 8. Используй их существующие трассы и отчёт. Генератор подтверждает тракт, а проверку реального датчика угла нужно выполнить отдельно.

Читать выборочно: [Lee/Seshia, Introduction to Embedded Systems, гл. 3 Discrete Dynamics](#) — состояние и отсчёты; [ESP-IDF PCNT: Introduction, Channel Actions](#) — смысл счёта A/B. API сверяй с закреплённой версией.